ENGINEERING LEARNING ACADEMY

Engineering Masterclass 002

Career Intelligence Copilot — FR-017 Agent Evaluation & Observability

Derive-Only Metrics, Reconstructability, and Anti-Theatre Observability

Functional Requirement	FR-017 — Agent Evaluation & Observability
Edition	Engineering Learning Academy — Lean Edition
Status	Complete / Frozen / Accepted
Audience	Experienced software engineers preparing for technical interviews
Document type	Official study edition (PDF)

Faithful rendering of the Lean Engineering Masterclass Markdown. Canonical engineering remains the FR acceptance report and ADR.

Contents

Executive Summary

FR-017 answers a deceptively simple question: once a multi-agent orchestration layer has written honest audit records, how do you *evaluate* those records without inventing an observability product?

The accepted answer is a **narrow derive-only evaluation substrate**. Existing `OrchestrationRun`, `Handoff`, and child `AgentRun` artefacts are treated as the only evidence. Pure functions derive metrics, correlate parent and child execution, and score reconstructability against twelve fixed questions (R1–R12). A thin read-only CLI presents the result. Nothing mutates the runtime. Nothing becomes a second system of record.

The commercial posture is intentionally modest: **low near-term product value, high learning and interview value**. Daily job preparation still prefers the simpler bounded agent path. Horizon 1B is not gated on this work. The engineering win is honesty under uncertainty — especially the distinction between *missing* metadata and *measured zero*, and the refusal to repair contradictory audits.

If you can explain FR-017 well in an interview, you can explain how mature systems separate **instrumentation**, **evaluation**, and **dashboards** — and why those three are not the same decision.

---

1. The Engineering Problem

After FR-016, Career Intelligence Copilot had orchestration audits: parent runs, typed handoffs, stop reasons, child agent runs, and resume/idempotency keys. What it lacked was a first-class way to *use* those audits as evaluation evidence.

An owner (or interviewer) needed to answer questions like:

1. Can I reconstruct the authority story of a run without reading source code?
2. Can I compare runs offline, deterministically, without a live LLM?
3. When tokens or cost are absent, is that "zero usage" or "unknown"?
4. If a child id does not join cleanly to a handoff, do we invent linkage or surface the gap?

Those are evaluation problems. They are not "add Prometheus" problems. They are not "build a Grafana board" problems. They are questions about **evidence**, **semantics**, and **source-of-truth boundaries**.

FR-017's real problem statement is therefore:

> Prove that existing orchestration audits are reconstructable and evaluable — without changing the orchestration runtime, without creating a telemetry store, and without pretending that dashboards equal understanding.

---

2. Why Previous Approaches Were Insufficient

Several tempting approaches were considered and rejected. Understanding the rejections is half the lesson.

2.1 The laundry-list "observability FR"

The original functional wording read like a catalogue: traces, checkpoints, browser journeys, dashboards, richer telemetry. Much of that catalogue was either already owned elsewhere (workflow traces in FR-008, truth validation in FR-014, BOPA metrics in FR-015, orchestration audits in FR-016) or was product theatre. Building the catalogue would have duplicated frozen work and delayed Horizon 1B for no acquisition gain.

2.2 Instrument the runtime to make metrics easy

Teams often "fix" evaluation by adding events until every chart is easy. That approach fails the derive-only gate. If your evaluation capability requires new runtime events to exist, you no longer have an evaluation of the *current* system — you have a redesign under measurement pressure. FR-017 forbids DOS, BOPA, OBS, Handoff, and AgentRun contract changes unless a genuine reconstructability blocker is proven. None was.

2.3 Fork child metrics inside the new module

BOPA already had `AgentRunMetrics` with careful missing-versus-zero semantics. Re-implementing token/cost/provider extraction inside FR-017 would create two competing definitions of the same facts. The chosen path reuses FR-015 helpers and rolls child metrics up only when the caller can supply them.

2.4 Dashboards as proof of observability

Dashboards create the *feeling* of control. They do not create reconstructability. A board that plots "latency = 0" for runs that never recorded latency is worse than no board: it converts absence into fake precision. FR-017 treats dashboard ambition as out of scope until derive-only evaluation demonstrably fails to answer owner-critical questions at scale.

2.5 Blocking the next product horizon on "completing observability"

Coupling recruiter outreach (Horizon 1B) to FR-017 would have inverted sequencing priorities. The application loop that unblocks 1B is discover → assess → prepare → review → submit → track (FR-008–FR-015). Evaluation substrate improves learning; it does not improve application throughput. The spike decision is normative: **FR-017 must not gate Horizon 1B**.

---

3. The Chosen Architecture

The architecture is deliberately boring. That is the point.

```
Existing OrchestrationRun / Handoff / child AgentRun
        ↓
Deterministic metric derivation
        ↓
R1–R12 reconstructability evaluation
        ↓
Read-only owner CLI
        ↓
Operational / learning insight
```

3.1 Ownership matrix

Concern	Owner	FR-017 role
Workflow traces	FR-008	None
Truth engine	FR-014	Cite status only
BOPA / AgentRun metrics	FR-015	Reuse
Orchestration audits	FR-016	Source records
Derive, correlate, evaluate, present	**FR-017**	Evaluation only

3.2 Derive-only module

`career_intelligence.multi_agent.observability` is a pure layer. It extracts:

- per-run metrics (identity, goal, status, stop reason, limits, observation snapshot)
- handoff metrics (allow/deny, policy fields, lifecycle)
- optional child roll-ups (provider, tokens, cost, latency — `None` when absent)
- parent/child correlation (including orphans and join gaps)
- reconstructability reports for R1–R12

No store writes. No supervisor start/resume. No specialist execution.

3.3 Reconstructability as acceptance, not narrative

R1–R12 are fixed questions that an owner should be able to answer from artefacts alone:

Id	Story beat
R1	Owner goal
R2	Observed state
R3	Specialist selection
R4	DelegationPolicy consistency
R5	Authority boundary (OBS / BOPA only)
R6	Handoff lifecycle
R7	Child output references
R8	Stop reason
R9	Owner next action
R10	Limits
R11	Parent/child walk
R12	Resume / idempotency

PASS means supported by evidence. **FAIL** names the missing or contradictory evidence. The evaluator never invents precision and never "heals" bad audits.

3.4 Thin read-only presentation

Owner operability is a CLI under the existing namespace:

- `cic agent orchestrate metrics` — one run (persisted id or fixture)
- `cic agent orchestrate metrics-corpus` — offline suite summary

Presentation is subordinate to derivation. The CLI is an interview and debug surface, not a product dashboard.

3.5 What was explicitly not built

No telemetry store. No new audit event kinds. No live tracing platform. No alerts. No pipeline mutation. No submission behaviour. No Horizon 1B integration.

3.6 Runtime Example

One conceptual evaluation flow (diagram seed):

```
Persisted orchestration audit
        ↓
Derive metrics (read-only)
        ↓
Score reconstructability (R1–R12)
        ↓
Surface gaps honestly (missing ≠ zero; orphans / contradictions)
        ↓
Owner insight — no runtime mutation
```

---

4. Engineering Principles

These principles are the transferable core of FR-017.

4.1 Derive before instrument

Prefer proving what existing records already support. Instrument only when reconstructability fails for a reason that cannot be fixed by better derivation or clearer ownership. This inverts the common "emit everything, figure it out later" habit.

4.2 Observability is not automatically dashboards

Observability, in this lesson, means: *can a competent engineer reconstruct what happened and what was unknown?* Charts are one possible presentation. They are not the definition.

4.3 Missing ≠ zero

Optional provider, model, token, cost, and latency fields use `None` for absence and `0` / `0.0` for measured emptiness. Counts may legitimately be zero when emptiness is measurable (zero handoffs). Coercing missing → zero is a semantic bug that infects every aggregate downstream.

4.4 Detect contradictions; do not repair them

Orphan child ids and contradictory parent/handoff linkage produce R-signal failures. Evaluation surfaces them. It does not rewrite audits to look healthy. Repair belongs to runtime correctness work, not to an evaluation layer pretending success.

4.5 Preserve source-of-truth boundaries

FR-017 evaluates. It does not become the system of record. FR-015 owns child metrics schemas. FR-016 owns orchestration audit shape. Crossing those boundaries creates twin definitions and silent drift.

4.6 Read-only evaluation surfaces

An evaluation CLI that can start supervisors or mutate pipelines is not an evaluation CLI; it is a control plane with a misleading name. FR-017 paths load only.

4.7 Do not let learning work block acquisition work

Narrow evaluation substrate must not gate recruiter outreach. Sequencing discipline is an engineering principle, not only a roadmap preference.

Why Employers Care

Employers need engineers who can evaluate complex systems without inventing parallel platforms. This work demonstrates transferable judgment: derive evidence from existing records, refuse false precision when data is missing, detect parent/child inconsistency without silently "fixing" it, and keep evaluation read-only so diagnosis cannot become accidental mutation. That is production-grade evaluation discipline — useful anywhere audits, workflows, or multi-service traces must be trusted.

---

5. Validation

Validation Summary

Item	Result
Major outcome	Offline reconstructability **GO** without runtime instrumentation
Corpus	**15/15** deterministic cases; repeatability confirmed
Recommendation	**ACCEPT AND FREEZE** — remain narrow derive-only
Constraints	No DOS/BOPA/OBS changes; no dashboards/telemetry SoT; Horizon 1B unblocked; prefer simpler daily prep path

Validation was designed to prove reconstructability *without* live orchestration.

5.1 Deterministic corpus (15/15 GO)

An offline corpus of fifteen fixtures covers the shapes that matter:

- successful orchestration and blocked delegation
- BOPA child and OBS brief, including `prepare_then_brief`
- missing optional metadata versus measured zero
- orphan child, missing child join, stale handoff
- resumed / idempotent run
- loop stops (repeated, circular, no-progress)
- provider unavailable
- contradictory audit
- mixed aggregate roll-up

Expected failures are part of the design: orphan cases fail R11; contradictory audits fail R7 and R11. A suite that only passes happy paths would be reconstructability theatre.

The harness runs twice and compares for deterministic repeatability. Result: **15/15 PASS**, `go_no_go=GO`, deterministic repeat **True**, no runtime instrumentation required.

5.2 Contract and CLI tests

Unit suites lock:

- derive contracts and missing-versus-zero behaviour
- corpus expectations including intentional R failures
- CLI presentation and read-only store behaviour
- regressions against FR-015 / FR-016 CLI surfaces

5.3 Owner manual validation

Manual demos prove the owner workflow: persisted run reconstruction, fixtures for missing/zero/orphan/contradiction, full corpus summary, and byte-identical proof that metrics commands do not mutate stored JSON. A complete reconstruct story follows:

goal → observed state → specialist selection → delegation → handoff lifecycle → child output → stop reason → owner next action → limits → resume/idempotency → R1–R12.

5.4 What "validated" means here

Validated does **not** mean "commercially adopted as daily default." It means: given the audits we already have, reconstructability is reliable, semantics are honest, and evaluation remains read-only.

---

6. Trade-offs

Accepted

Choice	Why it was worth it
Derive-only over existing audits	Tests the real system, not a redesigned one
R1–R12 as first-class acceptance	Forces evidence, not storytelling
Reuse FR-015 child metrics	One definition of tokens/cost/provider
Thin CLI instead of UI	Interviewable and owner-operable without product weight
Low commercial claim	Avoids observability theatre and false ROI

Deferred

Token/cost completeness for every offline fixture, CLI polish, and richer fault-injection productisation. Nulls remain honest; polish is not a freeze blocker.

Future FR (only if justified)

Richer time-travel replay or broader observability — **only** when derive-only evaluation fails to answer owner-critical questions at scale. Desire for dashboards is not justification.

Out of scope

Dashboards, alerts, live tracing platforms, telemetry backends, monitoring SaaS, pipeline mutation, submission behaviour, Horizon 1B integration.

Honest product assessment

Question	Answer
Improves daily preparation?	No material improvement
Materially improves Horizon 1B?	No
Useful operational / audit insight?	Yes
Justifies dashboards now?	No
Should remain narrow?	Yes

This honesty is itself an engineering artefact. Overclaiming value is a form of technical debt in product communication.

---

7. Interview Preparation

Use these as spoken answers, not memorised paragraphs.

Q: What problem did FR-017 solve?
A: How to evaluate multi-agent orchestration audits honestly — reconstructability, missing-data semantics, and parent/child correlation — without building an observability platform or changing the runtime.

Q: Why not add dashboards first?
A: Dashboards present numbers; they do not create trustworthy semantics. If missing latency is plotted as zero, the board lies. We proved derive-only reconstructability first. Dashboards remain unjustified until that approach fails at scale.

Q: What does derive-only mean in practice?
A: Pure functions over existing `OrchestrationRun` / `Handoff` / child `AgentRun` records. No new events, no telemetry store, no DOS/BOPA/OBS behaviour changes.

Q: Explain missing versus zero.
A: `None` means the optional field was never recorded. `0` means it was recorded and measured empty. Aggregates must not coerce absence into zero or every cost roll-up becomes fiction.

Q: What are R1–R12?
A: Twelve reconstructability questions from goal through resume/idempotency. PASS requires evidence; FAIL names the gap. Contradictions are surfaced, not repaired.

Q: Give an example of an expected FAIL.
A: An orphaned child reference fails R11. A contradictory completed run that cannot join child output fails R7 and R11. The corpus treats those as GO when detected correctly.

Q: Why reuse FR-015 metrics instead of redefining them?
A: Child token/cost/provider semantics already existed. Forking them would create two truths. Evaluation should consume authoritative metrics, not re-author them.

Q: Is this the daily operator path?
A: No. Ordinary preparation remains the simpler bounded agent command. FR-017 is learning/substrate and audit evaluation.

Q: Did this block the next product horizon?
A: Explicitly no. Application-loop usability unblocks Horizon 1B, not completion of every learning FR.

Q: When would you build richer observability?
A: When owner-critical questions cannot be answered from derived audits at the scale that matters — with concrete failing evidence, not aesthetic preference.

Q: What would you say is the main risk this design avoids?
A: Observability theatre: instrumenting and charting until the system looks monitored while reconstructability and semantic honesty remain unproven.

---

8. Three Engineering Lessons to Remember

1. **Derive before you instrument.**
   Prove what existing records already support. New telemetry is a design change, not a default first step.
2. **Missing is not zero — and contradictions are not repaired by evaluation.**
   Honest semantics beat pretty aggregates. An evaluation layer that heals bad audits is lying for convenience.
3. **Observability is reconstructability under authority boundaries — not dashboards.**
   If you cannot walk goal → decision → handoff → child → stop → next action from artefacts, you do not have observability yet, no matter how many panels you ship.

Closing Frame

FR-017 is a small system with a large teaching surface. It freezes a posture that senior engineers are often asked to defend verbally:

- separate evaluation from runtime mutation
- refuse second sources of truth
- keep commercial claims proportional to evidence
- leave adjacent product horizons unblocked when learning work is not on the critical path

That is the interview-ready story. Not “we built observability,” but “we made orchestration audits evaluable — honestly, narrowly, and without theatre.”

Memorable Closing Statement

Derive the truth you already have — before you instrument a truth you wish you had.

Lean Edition Masterclass — generated for Engineering Learning Academy import (Gamma later). Does not replace the canonical acceptance report or ADR-009.